

Why did an extra coding task appear to improve the prediction?

A weak comparison task and incomplete final tests made this case look more convincing than it was.

1 FIRST ANSWER

The model gave two different answers

The model first wrote "True 3", then ended with "False 2". Two reviewers saw only this answer; both called it conflicting.

2 RELATED CODING TASK

Two names refer to the same set

The model returned ["read", "write"]. This was correct because permissions and granted refer to the same set. Its function passed one automatic test.

2 COMPARISON CODING TASK

A function changes a shared set

The model returned ["checked", "new"]. Its function also passed one automatic test. The task was meant to cover a different skill, but it still involved changing a shared set.

3 PREDICTIONS SAVED BEFORE THE FINAL TASK

Either passed task changed the prediction to "pass"

The rule started at 0.50, removed 0.10 for the conflicting answer, then added 0.20 for any coding task that passed.

FIRST ANSWER ONLY

0.40 -> "fail"

RELATED TASK

0.60 -> "pass"

COMPARISON TASK

0.60 -> "pass"

These numbers were chosen in advance; they are scores, not probabilities.

4 FINAL TASK, SHOWN AFTER THE PREDICTIONS

The final tests missed part of the task

The final task asked the model to remove a tag from the supplied list, then return the remaining tags in sorted order. Its answer passed both automatic tests.

WHAT THE TESTS MISSED

The model created a new list, so the supplied list did not change. The tests checked only the returned values; they never checked the supplied list.

Reviewers later agreed: the task or its tests had a problem.

5 WHAT THIS CASE SHOWS

This case cannot tell us whether the related task helped

The related task changed "fail" to "pass". The comparison task did the same, and the final tests accepted an incomplete solution. This case cannot tell us whether the related task helped.